

"E-VOTING SYSTEM USING BLOCKCHAIN AND PYTHON"

A Dissertation

Submitted in Partial fulfillment for the award of

**Bachelor of Technology in
Computer Science and Engineering
YEAR 2022-26**

Submitted to:



RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA BHOPAL (M.P.)

Submitted By:

**Sojwal Sonkusale
ENROLL. NO. 0905CS221200**

**Yogesh Yadav
ENROLL. NO. 0905CS221233**

Under the Guidance of:

**Ms. Bhawna Ojha
Assistant Professor**

Department of Computer Science and Engineering



**INSTITUTE OF TECHNOLOGY & MANAGEMENT GWALIOR (M.P.)
-475001**

CANDIDATE DECLARATION

We are **SOJWAL SONKUSALE** and **YOGESH YADAV** students of B.Tech. (Computer Science) VIII semester Roll No **0905CS221200** and **0905CS221233** hereby declare that the Project entitled **“E-VOTING SYSTEM USING BLOCKCHAIN AND PYTHON”** which is being submitted to Department of computer science & Engineering in ITM, Gwalior is our authentic work carried out in our VIII semesters.

We declare that our work has not been submitted in part or in full to any other university or institution for the award of any degree or diploma.

SOJWAL SONKUSALE
(0905CS221200)
YOGESH YADAV
(0905CS221233)

CERTIFICATE

This is to certify that the project entitled **“E-VOTING SYSTEM USING BLOCKCHAIN AND PYTHON”** was submitted by **Sojwal Sonkusale (0905CS221200)** and **Yogesh Yadav (0905CS221233)** in partial fulfillment of the requirement for the award of B. Tech. degree in Computer Science & Engineering to Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (M.P.) is a record of bonafide work done by them, under my guidance

HEAD OF DEPARTMENT:

Dr. Pradeep Yadav
Associate Professor
Department of CSE
ITM, Gwalior (M.P)

PROJECT GUIDE:

Ms. Bhawna Ojha
Assistant Professor
Department of CSE
ITM, Gwalior (M.P)

External Examiner

ACKNOWLEDGEMENT

The most awaited moment of successful completion of an endeavor is always a result of persons involved implicitly or explicitly there in and it is impossible without the help and guidance of the people around us. The successful completion of this thesis report is a result of dedicated efforts put in by many people and this report would have been incomplete without giving due credit to them. This acknowledgement is indeed a small token of gratitude in recognition of their help.

We express our gratitude to **Dr. Meenakshi Mazumdar**, Director, Institute of Technology and Management, Gwalior for her valuable suggestions. We feel highly privileged to express our sincere thanks and deepest sense of gratitude to our guide **Ms. Bhawna Ojha** who has spared her precious time & guided us to present this dissertation. We sincerely acknowledge her for extending her valuable guidance, support for literature, critical review of report & above all the moral support she has provided us at all stages of this dissertation.

We wish to thank our Head of the department **Dr. Pradeep Yadav**, for his support and encouragement. Our thanks are extended to all staff members and persons who directly or indirectly helped us in achieving our goals. We could not have completed this work without the guidance, confidence and encouragement of our parents. They deserve more credit than we can give for instilling in us a good work ethic and a desire to always learn more. Our thanks are also extended to all our friends who have kept our spirits high throughout this work.

Sojwal Sonkusale
Enroll. No. 0905CS221200
Yogesh Yadav
Enroll. No. 0905CS221233

ABSTRACT

The rapid advancement of digital technology has increased the demand for secure and transparent electronic voting systems. Traditional voting systems often suffer from issues such as vote tampering, lack of transparency, duplicate voting, and dependence on centralized authorities. To overcome these challenges, this project presents a **Secure Blockchain Voting System** developed using Python and Blockchain technology.

The proposed system uses blockchain as a decentralized and immutable ledger to securely store voting records. Each vote is encrypted using SHA-256 cryptographic hashing and stored inside blocks linked together through hash values. The system also implements a Proof-of-Work consensus mechanism to validate new blocks and maintain chain integrity. This ensures that votes cannot be modified once recorded, thereby increasing trust and transparency in the election process.

The project provides multiple interfaces including a Flask-based web application, a Tkinter desktop GUI, and a command-line server mode. Voter authentication is handled through unique voter IDs, preventing duplicate voting while maintaining voter anonymity. The election commission can securely access verified voting results constituency-wise through a dedicated commission portal.

The system demonstrates how blockchain technology can improve the security, reliability, and transparency of modern electronic voting systems. The proposed solution minimizes the risks associated with centralized voting systems and provides an efficient, tamper-proof, and auditable voting mechanism suitable for future digital elections.

TABLE OF CONTENT

Abstract	4
Table of Content	5
List of Figures	8
List of Tables	9
Chapter 1. Introduction	10
1.1 Introduction	11
1.2 Objectives	12
1.3 Problem Statement	13
1.4 Need of Blockchain in Voting System	13
1.5 Existing System	14
1.6 Limitations of Existing System	14
1.7 Proposed System	14
1.8 Scope of the Project	15
1.9 Advantages of the System	15
Chapter 2. Literature Survey	16
2.1 Introduction	17
2.2 Previous Work	17
2.3 Blockchain Technology	18
2.4 SHA-256 Hashing	19
2.5 Proof of Work Consensus	20
2.6 Research Gap	21
Chapter 3. System Study	22
3.1 Existing System Analysis	23

3.2 Proposed System Analysis	24
3.3 Feasibility Study	25
3.3.1 Technical Feasibility	25
3.3.2 Operational Feasibility	26
3.3.3 Economic Feasibility	27
3.4 System Requirements	27
3.4.1 Hardware Requirements	28
3.4.2 Software Requirements	29
Chapter 4. System Analysis & Design	30
4.1 Functional Requirements	31
4.2 Non-Functional Requirements	32
4.3 Software Requirement Specification (SRS)	33
4.4 System Architecture	34
4.5 System Flowchart	36
4.6 Data Flow Diagram (DFD)	37
4.6.1 Level 0 DFD	37
4.6.2 Level 1 DFD	38
4.7 UML Diagrams	39
4.7.1 Use Case Diagram	39
4.7.2 Class Diagram	40
4.7.3 Sequence Diagram	41
4.8 Database / Blockchain Structure	42
4.9 System Design	42
Chapter 5. Implementation & Testing	44
5.1 Introduction	45

5.2 Technologies Used	46
5.3 Project Modules	47
5.4 Blockchain Implementation	48
5.5 Web Application Implementation	49
5.6 GUI Implementation	49
5.7 API Implementation	50
5.8 Testing	51
5.8.1 Unit Testing	51
5.8.2 Integration Testing	52
5.8.3 System Testing	52
5.9 Screenshots of the Project	53
5.9.1 Home Page	53
5.9.2 Voting Interface	54
5.9.3 Blockchain View	55
5.9.4 Result Dashboard	56
5.9.5 Commission Login Page	57
Chapter 6. Results & Future Scope	62
6.1 Project Results	63
6.2 Advantages of the System	64
6.3 Limitations	65
6.4 Future Enhancements	66
6.5 Conclusion	67
References	68

List of Figure

Figure 4.1 System Architecture	34
Figure 4.2 System Flowchart	36
Figure 4.3 Level 0 DFD	37
Figure 4.4 Level 1 DFD	38
Figure 4.5 Use Case Diagram	39
Figure 4.6 Class Diagram	40
Figure 4.7 Sequence Diagram	41
Figure 5.1 Home Page Screenshot	53
Figure 5.2 Voting Interface Screenshot	54
Figure 5.3 Blockchain View Screenshot	55
Figure 5.4 Result Dashboard Screenshot	56
Figure 5.5 Commission Login Page Screenshot	57

List of Tables

Table 3.1 Hardware Requirements	28
Table 3.2 Software Requirements	29
Table 4.1 Fundamental Systemic User Classifications	33
Table 5.1 Technologies Used	46
Table 5.2 Fundamental API Endpoints	50

CHAPTER - 1

CHAPTER - 1

INTRODUCTION

1.1 INTRODUCTION

In the modern digital era, technology has transformed various sectors such as banking, education, healthcare, and communication. However, the voting system in many regions still relies on traditional methods that are often time-consuming, costly, and vulnerable to manipulation. Electronic voting systems were introduced to improve efficiency, but centralized e-voting systems still face serious security and transparency challenges.

One of the major concerns in existing voting systems is the possibility of vote tampering, duplicate voting, lack of transparency, and dependence on centralized authorities. These issues reduce public trust in the electoral process. Therefore, there is a need for a secure, transparent, and tamper-proof voting mechanism that ensures fairness and integrity in elections.

Blockchain technology provides an effective solution to these challenges. Blockchain is a decentralized and distributed ledger technology where data is stored in interconnected blocks secured using cryptographic algorithms. Once information is added to the blockchain, it becomes immutable and cannot be altered without network validation. This feature makes blockchain highly suitable for secure voting applications.

The proposed project, “**Secure Blockchain Voting System,**” is developed using Python programming language and blockchain concepts to create a secure electronic voting platform. The system uses SHA-256 cryptographic hashing and Proof-of-Work consensus mechanisms to maintain data integrity and ensure secure vote storage. Each vote is stored as a block in the blockchain, making the voting process transparent and tamper-proof.

The system provides multiple interfaces including a Flask-based web application, a desktop GUI using Tkinter, and a command-line server mode. Voters can cast their votes using unique voter IDs, while the election commission can securely view constituency-wise results through an authenticated portal.

This project demonstrates how blockchain technology can improve the reliability, transparency, and security of digital voting systems while eliminating many limitations of traditional voting methods.

1.2 OBJECTIVES

The **Secure Blockchain Voting System** is designed with the following key objectives:

- **Develop a secure and transparent e-voting platform** by leveraging blockchain technology, ensuring that every vote is recorded in a tamper-proof manner. This system aims to build trust among voters and authorities by providing a decentralized and verifiable voting process.
- **Prevent vote tampering** through immutable blockchain records. Once a vote is cast and stored in the blockchain, it cannot be altered or deleted, thereby safeguarding the integrity of the election process.
- **Eliminate duplicate voting** by implementing unique voter identification mechanisms. Each voter will be authenticated and allowed to cast only one vote, ensuring fairness and accuracy in the results.
- **Ensure voter anonymity** using SHA-256 cryptographic hashing. This guarantees that while votes are securely stored and verifiable, the identity of the voter remains confidential, protecting privacy.
- **Implement Proof-of-Work consensus** for block validation. This ensures that only legitimate blocks are added to the chain, maintaining the reliability and security of the voting records.
- **Provide transparency** by enabling blockchain verification. Election authorities, auditors, and even voters can verify the integrity of the voting process without compromising anonymity.
- **Design a user-friendly interface** for both web and desktop platforms, making the system accessible to a wide range of users regardless of their technical expertise.
- **Enable constituency-wise result analysis** for election authorities, allowing detailed insights into voting patterns and outcomes across different regions.
- **Demonstrate real-world blockchain implementation** using Python programming, showcasing how theoretical concepts can be applied in practice to solve critical challenges in democratic processes.

1.3 PROBLEM STATEMENT

Traditional voting systems suffer from several limitations such as vote manipulation, lack of transparency, centralized control, and duplicate voting. Existing electronic voting systems are also vulnerable to cyberattacks and unauthorized modifications due to centralized storage mechanisms.

The absence of a secure and decentralized system creates trust issues among voters and election authorities. Therefore, there is a need for a reliable voting platform that ensures data integrity, transparency, voter privacy, and secure vote storage.

The proposed blockchain-based voting system addresses these challenges by implementing decentralized blockchain technology, cryptographic hashing, and Proof-of-Work validation mechanisms to create a secure and tamper-proof election system.

1.4 NEED OF BLOCKCHAIN IN VOTING SYSTEM

Blockchain technology is essential in voting systems because it provides decentralization, transparency, and immutability. In traditional systems, data is stored in centralized servers, which can be vulnerable to hacking or unauthorized access. Blockchain removes this single point of failure by distributing data across multiple blocks connected through cryptographic hashes.

The key needs of blockchain in voting systems include:

- Secure storage of votes
- Prevention of vote tampering
- Transparent verification process
- Decentralized data management
- Improved voter trust
- Elimination of duplicate voting
- Easy auditability of election records

By using blockchain, every vote becomes permanent and verifiable, ensuring a fair and transparent election process.

1.5 EXISTING SYSTEM

The existing voting systems mainly include:

1. Traditional Paper Ballot System

In this system, voters cast votes manually using paper ballots. The counting process is time-consuming and prone to human errors.

2. Electronic Voting Machines (EVMs)

Electronic Voting Machines are used to automate the voting process. Although faster than paper ballots, they still rely on centralized control and lack complete transparency.

3. Online Voting Systems

Some systems allow online voting through centralized servers. These systems are vulnerable to cyberattacks, data manipulation, and unauthorized access.

1.6 LIMITATIONS OF EXISTING SYSTEM

The limitations of existing voting systems are:

- Risk of vote tampering
- Lack of transparency
- Duplicate voting possibilities
- Centralized database vulnerabilities
- Difficult audit process
- High operational costs
- Slow result generation
- Dependence on human intervention
- Security risks in online systems

These limitations reduce voter confidence and create concerns regarding election fairness.

1.7 PROPOSED SYSTEM

The proposed Secure Blockchain Voting System uses blockchain technology to create a decentralized and tamper-proof voting platform.

In this system:

- Each vote is stored as a block in the blockchain.

- SHA-256 hashing secures voter information.
- Proof-of-Work validates each new block.
- Duplicate voting is prevented using voter ID verification.
- Blockchain integrity is continuously verified.
- Election results are automatically generated constituency-wise.

The system provides multiple interfaces such as web application, desktop GUI, and CLI server mode for flexibility and accessibility.

1.8 SCOPE OF THE PROJECT

The scope of this project includes:

- Development of a blockchain-based voting platform
- Secure vote storage using cryptographic hashing
- Real-time blockchain verification
- Constituency-wise voting and result analysis
- Prevention of duplicate voting
- Web and desktop application support
- Educational demonstration of blockchain implementation

The project can be further expanded for national-level elections with advanced security measures and distributed blockchain networks.

1.9 ADVANTAGES OF THE SYSTEM

The advantages of the proposed system are:

- High security through blockchain technology
- Transparent and auditable election process
- Prevention of vote tampering
- Elimination of duplicate voting
- Decentralized architecture
- Faster vote counting and result generation
- Reduced operational costs
- Improved voter trust and confidence
- Immutable vote records
- Easy verification of blockchain integrity

CHAPTER - 2

CHAPTER - 2

LITERATURE SURVEY

2.1 INTRODUCTION

A literature survey represents a pivotal phase in project development, facilitating a comprehensive understanding of extant technologies, methodologies, and scholarly contributions pertinent to the proposed architecture. Within the domain of electronic voting, researchers and organizations have pioneered diverse mechanisms aimed at enhancing transparency, security, and systemic reliability. Nevertheless, a multitude of traditional electronic voting frameworks continues to grapple with critical impediments, including vote manipulation, opaque processes, centralized authority, and inherent security vulnerabilities.

Blockchain technology has surfaced as a robust paradigm to mitigate these foundational challenges. By offering decentralization, immutability, transparency, and advanced cryptographic security, it serves as an ideal substrate for electronic voting applications. Numerous investigators have advocated for **Blockchain-based voting systems** utilizing cryptographic algorithms, smart contracts, and distributed consensus protocols to facilitate secure and tamper-proof electoral processes.

This chapter delineates a meticulous study of previous endeavors and technologies associated with blockchain-enabled voting infrastructures. It further examines the specific algorithms and technical frameworks employed in secure voting applications, encompassing **SHA-256 hashing** and **Proof-of-Work consensus** mechanisms. The evaluative analysis of existing frameworks assists in identifying critical research gaps, thereby providing the impetus for the development of the proposed Secure Blockchain Voting System.

2.2 PREVIOUS WORK

The historical evolution of electronic voting has seen numerous efforts aimed at digitizing the electoral process. Initial frameworks primarily leveraged centralized database architectures, wherein voting data was stored and managed by a singular administrative authority. While such systems successfully mitigated the labor-intensive nature of manual counting, they

simultaneously introduced significant security vulnerabilities, including unauthorized data access, malicious database manipulation, and a profound lack of procedural transparency.

Subsequent research focused on the integration of cryptographic techniques to fortify the security of voting infrastructures. Sophisticated encryption algorithms were deployed to safeguard voter confidentiality and protect sensitive information; however, these centralized paradigms remained susceptible to single-point failures and targeted cyberattacks.

The emergence of blockchain technology has catalyzed the development of decentralized voting architectures. These frameworks utilize distributed ledgers to record votes within interconnected blocks, where each segment is cryptographically linked to its predecessor via secure hashes. The inherent immutability of blockchain ensures that voting records remain resistant to unauthorized modification, thereby fostering systemic transparency and trust.

Contemporary implementations have explored the use of **Ethereum smart contracts** for automated tallying and private blockchain networks employing custom consensus protocols. These advancements have effectively demonstrated the capacity for blockchain to eradicate vote tampering and enhance the perceived integrity of the democratic process.

Notwithstanding these technological strides, many extant blockchain-enabled systems are hampered by excessive computational overhead, deployment complexities, and limited scalability. The proposed research addresses these critical gaps by engineering a lightweight, secure, and user-centric blockchain voting platform utilizing **Python** and the **Flask** framework.

2.3 BLOCKCHAIN TECHNOLOGY

Blockchain represents a decentralized paradigm of digital ledger technology, engineered to facilitate the secure and distributed storage of data across a network of systems. This architectural framework consists of a sequential chain of blocks, wherein each constituent element encapsulates transactional data, temporal metadata, cryptographic hash signatures, and a reference to the cryptographic hash of its immediate predecessor.

The fundamental attributes characterizing blockchain technology encompass:

- **Decentralization**
- **Transparency**

- **Immutability**
- **Security**
- **Distributed consensus**

Within the context of a blockchain-enabled voting architecture, each discrete vote is processed as a transaction and permanently recorded within a block. Upon the integration of a block into the ledger, the stored information becomes exceptionally resistant to retrospective modification. This mechanism ensures that the voting records remain inherently secure and impervious to unauthorized tampering.

The proposed architecture leverages blockchain to facilitate the secure archival of electoral records and to provide a verifiable substrate for maintaining systemic integrity.

2.4 SHA-256 HASHING

SHA-256 (Secure Hash Algorithm 256-bit) constitutes a cryptographic hashing primitive extensively utilized in blockchain frameworks to ensure data integrity. It functions by transforming input data into a deterministic, fixed-length 256-bit hash signature.

The salient cryptographic properties of SHA-256 include:

- **One-way encryption**
- **Fixed output size**
- **Collision resistance**
- **High security**

In the developed system, the **SHA-256** hashing algorithm is implemented to:

- Encrypt and obfuscate voter identifications
- Compute unique block hashes
- Preserve the sequential integrity of the chain
- Authenticate the validity of individual blocks

The sensitivity of the algorithm ensures that even infinitesimal alterations to the input payload yield vastly disparate hash values, thereby fortifying the blockchain against malicious tampering attempts.

2.5 PROOF OF WORK CONSENSUS

Proof of Work (PoW) represents a robust consensus protocol deployed to validate transactions and facilitate the addition of new blocks to the distributed ledger. This mechanism necessitates the resolution of computationally intensive mathematical challenges to produce a hash signature that conforms to predefined criteria.

The primary functional objectives of the **Proof of Work** mechanism include:

- Mitigating unauthorized block generation
- Ensuring the cryptographic integrity of the blockchain
- Defending the network against adversarial attacks
- Sustaining the decentralized nature of the ledger

In the proposed electoral system, the **Proof-of-Work** protocol ensures that each candidate block satisfies specific hash constraints before being committed to the chain. This validation phase significantly enhances the security and overall reliability of the digital voting process.

2.6 RESEARCH GAP

A comprehensive evaluative analysis of existing voting infrastructures and blockchain-based implementations revealed several critical research gaps:

- A significant number of contemporary systems exhibit prohibitive complexity and implementation difficulties.
- Certain blockchain implementations necessitate substantial capital expenditure on infrastructure.
- Multiple frameworks lack intuitive and user-centric interfaces for voters.
- Extant systems often grapple with scalability bottlenecks and performance optimization issues.
- Deficiencies in transparent verification mechanisms for election results remain prevalent.
- The absence of multi-platform support (Web, GUI, CLI) limits the accessibility of many existing applications.

To effectively mitigate these constraints, the proposed **Secure Blockchain Voting System** incorporates:

- A lightweight and optimized blockchain architecture
- Diversified user interfaces including Web, GUI, and CLI modalities
- Advanced cryptographic security leveraging SHA-256 hashing
- Rigorous Proof-of-Work validation protocols
- Real-time capabilities for blockchain verification
- Constituency-specific electoral management utilities
- Transparent and immutable archival of voting records

This project endeavors to deliver a secure, transparent, and pragmatic blockchain-centric solution, serving as a functional prototype for academic and research-oriented applications in electronic democracy.

CHAPTER - 3

CHAPTER - 3

SYSTEM STUDY

3.1 EXISTING SYSTEM ANALYSIS

Conventional electoral frameworks predominantly utilize paper-based ballots or centralized electronic voting infrastructures. While these methodologies remain extensively deployed, they are characterized by profound operational constraints and security vulnerabilities. In the context of paper-centric systems, the manual aggregation of electoral data necessitates significant temporal and human resources, while inherent counting inaccuracies further compromise the precision of final outcomes.

Although centralized electronic paradigms enhance computational velocity and procedural efficiency, they introduce significant cybersecurity risks. The reliance on a singular, centralized database architecture creates a critical single point of failure, enabling adversarial entities to target isolated servers for the purpose of malicious vote manipulation or systemic disruption. Furthermore, the absence of comprehensive transparency within these frameworks precludes voters from independently verifying the integrity of their recorded selections.

The foundational impediments associated with contemporary voting architectures encompass:

- Inherent risk of unauthorized vote tampering
- Deficiencies in procedural transparency
- Susceptibility to duplicate voting practices
- Centralized administrative control and single-point failure vulnerabilities
- Protracted temporal requirements for electoral tallying
- Insufficient auditability and verification mechanisms
- Critical security weaknesses within server and database environments

Mitigating these constraints necessitates the engineering of a secure and transparent digital voting solution capable of ensuring systemic trust, procedural integrity, and functional reliability during electoral processes.

3.2 PROPOSED SYSTEM ANALYSIS

The engineered **Secure Blockchain Voting System** is architected to effectively mitigate the foundational constraints of conventional electoral frameworks through the strategic integration of blockchain technology. This paradigm facilitates the archival of votes within discrete, interconnected blocks, cryptographically linked via secure hash signatures. Upon committing electoral data to the ledger, the records attain inherent immutability, ensuring that once recorded, they remain impervious to retrospective modification or unauthorized deletion.

The proposed architecture implements the **SHA-256** cryptographic algorithm for the obfuscation of voter metadata and a rigorous **Proof-of-Work** consensus protocol to authenticate blocks prior to their integration into the chain. These mechanisms collectively fortify the system against adversarial manipulation while fostering unparalleled procedural transparency and systemic integrity.

To enhance functional accessibility, the infrastructure provides three distinct operational interfaces:

- **Flask-based Web Application**
- **Tkinter Desktop GUI**
- **Command-Line Interface (CLI)**

The electoral workflow is initiated upon the submission of a verified voter identification. Contingent upon the specific constituency, the voter selects their preferred candidate and commits their selection to the ledger. This vote is subsequently encrypted and archived within a blockchain block following successful mining operations and rigorous cryptographic validation.

The primary functional advantages characterizing the proposed system encompass:

- Tamper-proof and immutable electoral record archival
- Robust and secure protocols for voter authentication
- Comprehensive mitigation of duplicate voting practices
- Transparent and verifiable blockchain integrity
- Real-time computational aggregation of electoral results
- Decentralized and cryptographically secure architectural framework

The system represents an optimized, lightweight solution engineered to demonstrate the pragmatic application of blockchain-centric secure voting mechanisms within a digital democratic context.

3.3 FEASIBILITY STUDY

A feasibility study represents a critical evaluative phase to ascertain the pragmatic viability, economic sustainability, and technical implementability of the proposed architectural framework. This phase facilitates a comprehensive multi-dimensional assessment of project parameters prior to the commencement of functional development.

The feasibility analysis for the Secure Blockchain Voting System encompasses the following fundamental domains:

- **Technical Feasibility**
- **Operational Feasibility**
- **Economic Feasibility**

3.3.1 TECHNICAL FEASIBILITY

Technical feasibility involves a rigorous evaluative analysis to determine the availability of requisite technological frameworks, computational utilities, and developmental resources essential for the successful engineering of the proposed architecture.

The articulated project is deemed technically viable based upon the following foundational parameters:

- The **Python** ecosystem offers sophisticated libraries specifically engineered for robust blockchain development.
- The **Flask** micro-framework facilitates the deployment of lightweight and computationally efficient web-based applications.
- Cryptographic **SHA-256** hashing primitives are readily accessible via the native Python hashlib module.
- The architectural design necessitates minimal hardware requirements, precluding the need for high-end computational infrastructure.

- Pragmatic blockchain implementation is achievable through the utilization of standardized programming methodologies.
- The **Tkinter** library provides comprehensive support for the creation of native desktop graphical user interfaces.
- Data representation and systemic communication are optimized through the integration of the **JSON** format.

The integrated technological stack utilizes open-source, industrially reliable, and pragmatically implementable software components.

3.3.2 OPERATIONAL FEASIBILITY

Operational feasibility assesses the systemic capacity to function effectively within practical environments and ensures that end-users can seamlessly engage with the developed modalities.

The proposed architecture is operationally sustainable due to the following attributes:

- The deployment of a highly intuitive and user-centric graphical interface.
- Electoral participation is characterized by high procedural velocity and functional efficiency.
- The framework incorporates autonomous protocols for the continuous verification of blockchain integrity.
- Systemic transparency is maintained through the automated aggregation of electoral outcomes.
- Rigorous mitigation of duplicate voting practices significantly enhances electoral reliability.
- Administrative governance is facilitated through an authenticated commission portal.

The system is ideally suited for academic, research-oriented, and pedagogical demonstrations without necessitating convoluted operational workflows.

3.3.3 ECONOMIC FEASIBILITY

Economic feasibility analysis determines the fiscal viability of the project by assessing whether development can be concluded within acceptable capital constraints.

The engineered solution is economically sustainable based on the following criteria:

- The developmental tools and software utilities utilized are entirely open-source.
- Fundamental frameworks including **Python**, **Flask**, and **Tkinter** preclude the necessity for prohibitive licensing expenditure.
- System development is facilitated utilizing standard, pre-existing computational hardware.
- The architecture eliminates the requirement for capital-intensive specialized infrastructure.
- Systemic maintenance and deployment overhead remain negligible.

Consequently, the aggregate implementation expenditure of the proposed system remains significantly lower than that of conventional, large-scale electoral infrastructures.

3.4 SYSTEM REQUIREMENTS

System requirements delineate the foundational hardware and software infrastructures essential for the efficacious deployment and functional execution of the proposed architectural framework.

The requisite parameters are categorized into the following pivotal domains:

- **Hardware Requirements**
- **Software Requirements**

3.4.1 HARDWARE REQUIREMENTS

The minimum hardware requirements for the system are:

Component	Specification	Purpose
Processor	Intel Core i3 or above	To handle SHA-256 cryptographic hashing and Proof-of-Work computations efficiently.

Component	Specification	Purpose
RAM	4 GB minimum	Ensures smooth execution of the Flask web server and Tkinter GUI during concurrent operations.
Storage	500 MB free space	Provides space for storing the decentralized blockchain ledger and application code locally.
Display	Standard Monitor	Required for users and election commission to interact with the visual voting interfaces.
Internet	Required for Web Interface	Enables connectivity for the Flask-based web application and real-time blockchain verification.

3.4.2 SOFTWARE REQUIREMENTS

The software requirements for the project are:

Software	Specification	Purpose
Python 3.7+	Programming Language	Essential for core blockchain implementation, cryptography (SHA-256), and system logic.
Flask	Web Framework	Used for developing the lightweight web application interface and handling API requests.
Tkinter	Desktop GUI	Facilitates the creation of a native, user-friendly desktop interface for the application.

Software	Specification	Purpose
VS Code / PyCharm	Development Environment	Recommended IDEs for efficient coding, debugging, and project management during development.
Git	Version Control	Required for tracking changes, managing collaboration, and maintaining the project codebase.
Web Browser	Access Web Interface	Necessary for users and the Election Commission to access and interact with the Flask-based platform.
Operating System	Windows / Linux / macOS	The system is designed for cross-platform execution and deployment across major operating systems.

The proposed system can run efficiently on multiple operating systems and supports cross-platform execution.

CHAPTER - 4

CHAPTER 4

SYSTEM ANALYSIS & DESIGN

4.1 FUNCTIONAL REQUIREMENTS

Functional requirements describe the core functionalities and operations performed by the Secure Blockchain Voting System.

The major functional requirements of the system are as follows:

1. Voter Authentication

- The system must allow voters to enter a unique voter ID.
- The system must validate the voter ID before voting.
- Each voter should be allowed to vote only once.

2. Candidate Selection

- The system must display candidates based on constituency.
- Voters must be able to select one candidate from the available list.

3. Vote Casting

- The system must securely record votes.
- Vote data must be encrypted using SHA-256 hashing.
- A new block must be generated for each valid vote.

4. Blockchain Management

- The system must maintain blockchain integrity.
- Each block must contain:
 - Index
 - Timestamp
 - Vote data
 - Previous hash
 - Current hash
 - Proof value

5. Proof-of-Work Validation

- The system must validate blocks using Proof-of-Work.
- Only verified blocks should be added to the blockchain.

6. Result Generation

- The system must automatically count votes.
- Constituency-wise election results should be displayed.

7. Commission Login

- Election commission should access verified results using login credentials.
- Unauthorized access must be restricted.

8. Blockchain Verification

- Users should be able to verify blockchain integrity.
- The system must detect tampering attempts.

4.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality attributes and performance standards of the system.

1. Security

- The system must protect vote data using cryptographic hashing.
- Blockchain records should remain immutable.
- Unauthorized access should be prevented.

2. Reliability

- The system must provide accurate vote recording and counting.
- Blockchain verification should ensure data consistency.

3. Performance

- The system should process votes efficiently.
- Results should be generated quickly.

4. Scalability

- The system should support future expansion for larger elections.

5. Usability

- Interfaces should be user-friendly and easy to operate.

6. Availability

- The system should remain operational during election processes.

7. Maintainability

- The project should support easy updates and modifications.

8. Portability

- The application should run on multiple operating systems.

4.3 SOFTWARE REQUIREMENT SPECIFICATION (SRS)

Software Requirement Specification (SRS) defines the complete behavior and functionality of the proposed system.

Purpose

The purpose of this project is to develop a secure, transparent, and decentralized voting system using blockchain technology.

Scope

The system allows voters to cast secure votes and enables election authorities to verify results through blockchain validation.

Fundamental Systemic User Classifications

User Category	Functional Role
Voter	Facilitates the secure and transparent submission of electoral selections to the decentralized ledger.
Election Commission	Provides administrative oversight and deterministic verification of cryptographically authenticated results.
Administrator	Ensures continuous operational stability, systemic maintenance, and technical governance of the architecture.

Functional Modules

- Voter Authentication Module
- Candidate Management Module
- Blockchain Module
- Voting Module
- Result Management Module
- Commission Verification Module

Constraints

- Internet connection required for web interface
- Voter must possess valid voter ID
- Python environment must be installed

Assumptions

- Users have basic knowledge of system operation
- Election data is securely managed

4.4 SYSTEM ARCHITECTURE

The Secure Blockchain Voting System follows a layered architecture consisting of user interfaces, application logic, blockchain processing, and data storage.

Architecture Components

1. User Interface Layer

Provides interaction between users and the system.

Interfaces include:

- Flask Web Interface
- Tkinter GUI
- CLI Server Interface

2. Application Layer

Handles:

- Voter validation
- Candidate selection
- Vote processing
- Result generation

3. Blockchain Layer

Responsible for:

- Block creation
- Hash generation
- Proof-of-Work validation
- Chain verification

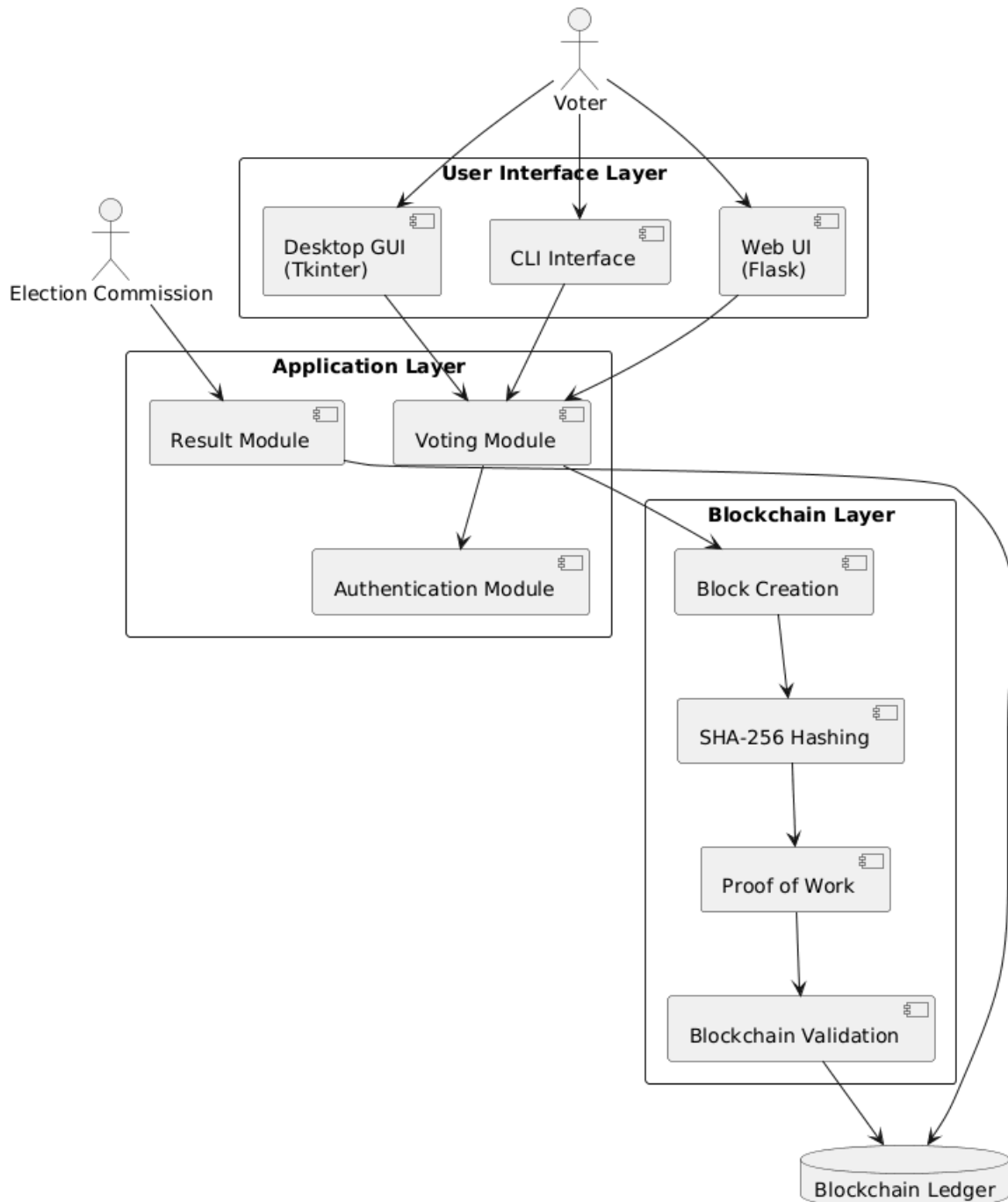
4. Data Layer

Stores:

- Blockchain records
- Vote transactions
- Candidate information

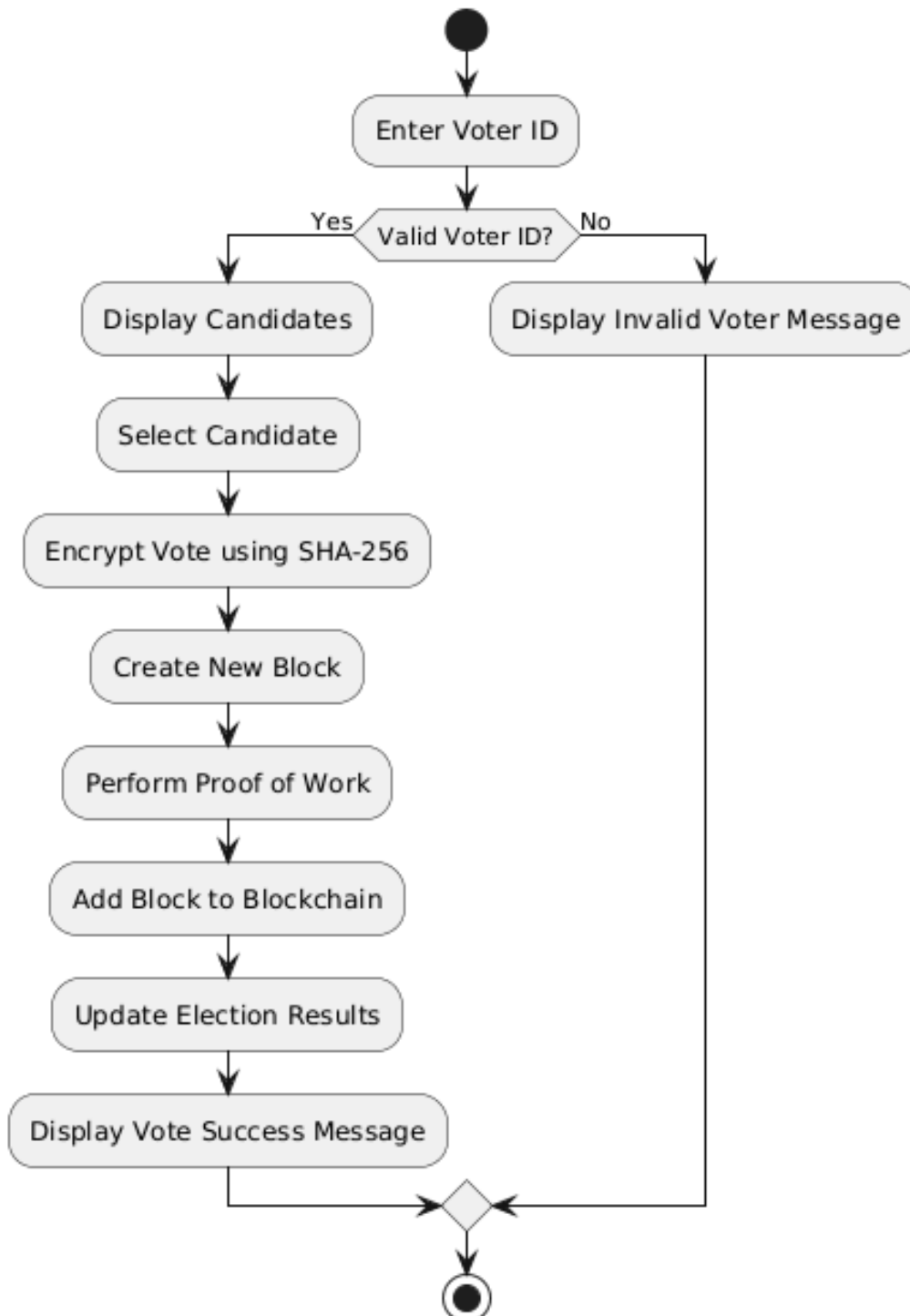
System Architecture Flow

Secure Blockchain Voting System - Architecture Diagram



4.5 SYSTEM FLOWCHART

The system flowchart represents the overall working process of the voting application.



4.6 DATA FLOW DIAGRAM (DFD)

Data Flow Diagram (DFD) represents the movement of data between different system components.

The Secure Blockchain Voting System contains:

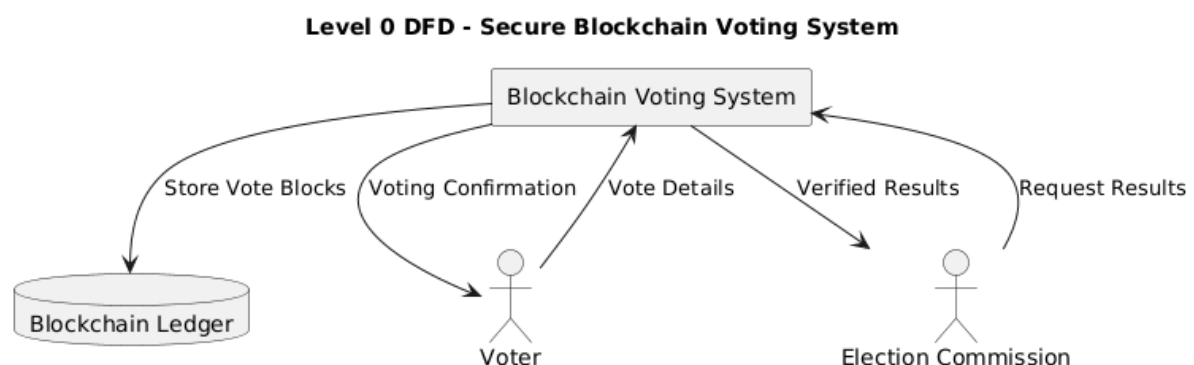
- Voter input
- Vote processing
- Blockchain verification
- Result generation

4.6.1 LEVEL 0 DFD

Level 0 DFD provides a high-level overview of the entire system.

Main Entities

- Voter
- Blockchain Voting System
- Election Commission



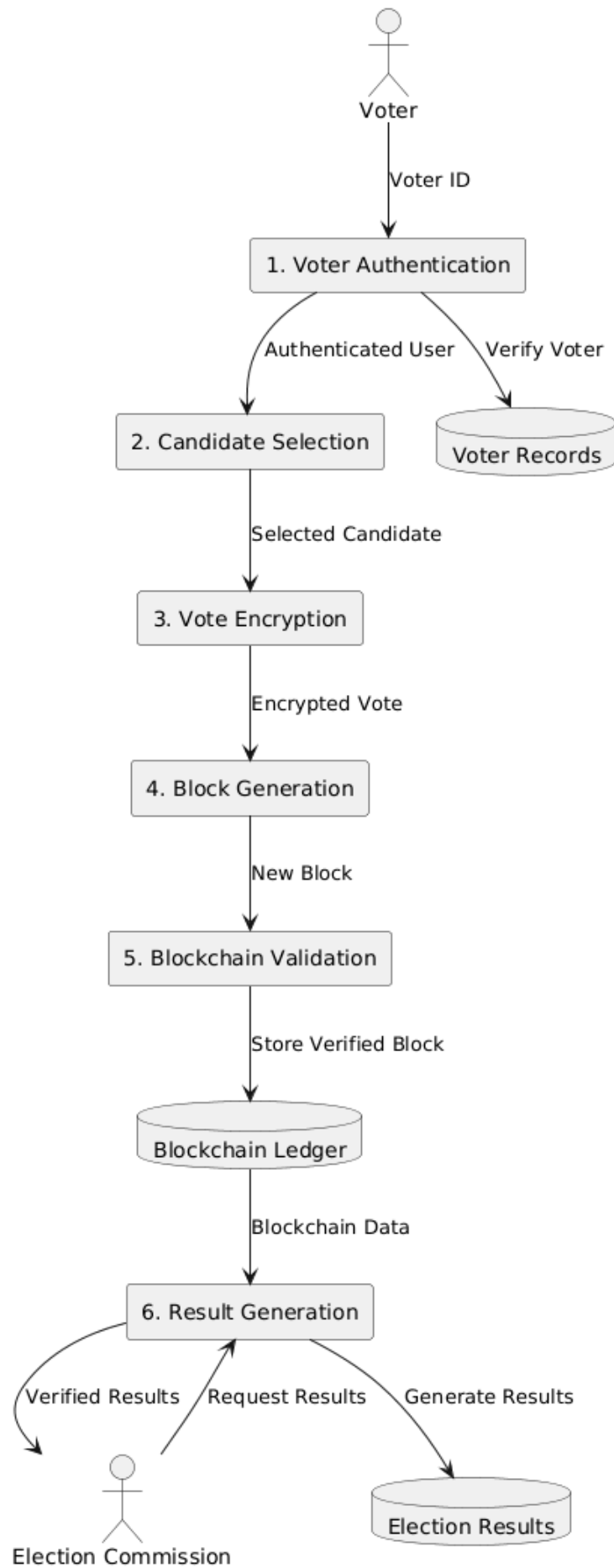
4.6.2 LEVEL 1 DFD

Level 1 DFD provides a detailed representation of internal processes.

Processes Included

1. Voter Authentication
2. Candidate Verification
3. Vote Encryption
4. Block Generation
5. Blockchain Validation

Level 1 DFD - Secure Blockchain Voting System



4.7 UML DIAGRAMS

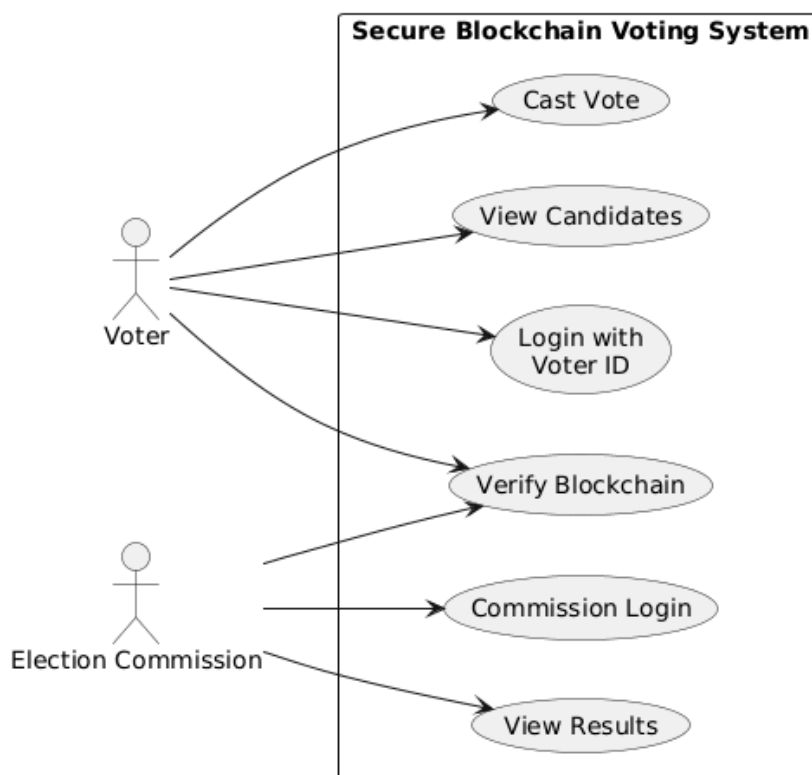
Unified Modeling Language (UML) diagrams visually represent the structure and behavior of the system.

The UML diagrams used in this project are:

- Use Case Diagram
- Class Diagram
- Sequence Diagram

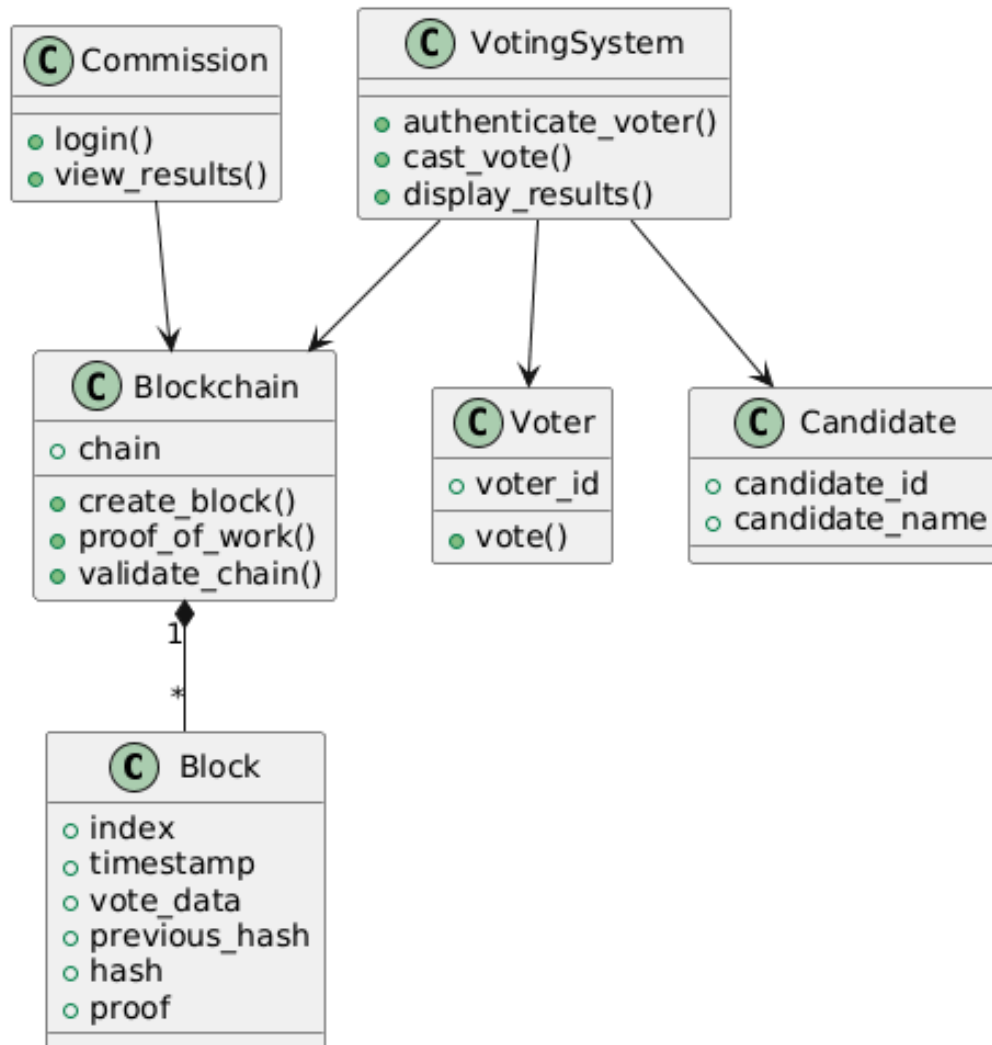
4.7.1 USE CASE DIAGRAM

The Use Case Diagram represents interactions between users and the system.



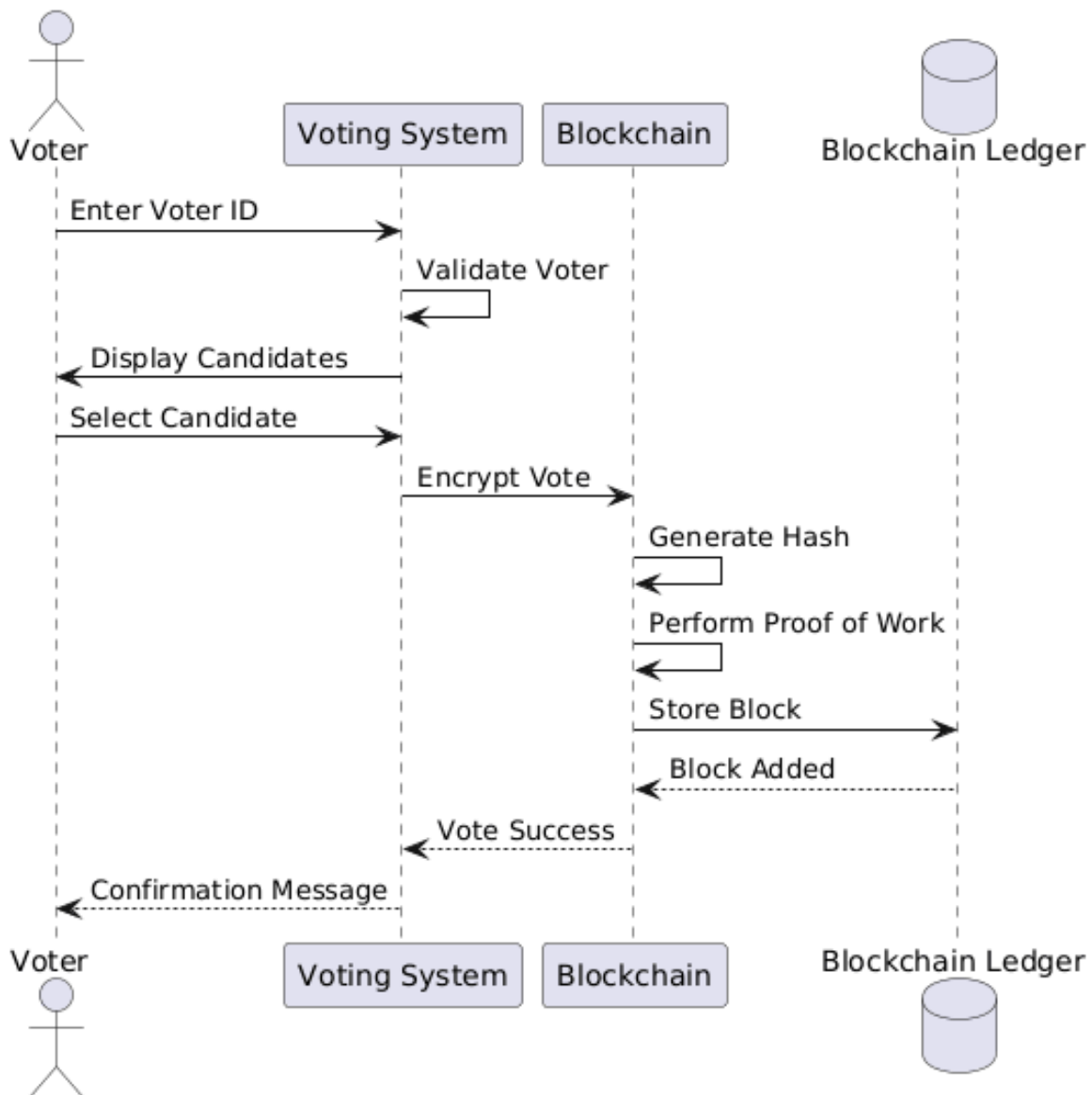
4.7.2 CLASS DIAGRAM

The Class Diagram represents the structure of classes and their relationships.



4.7.3 SEQUENCE DIAGRAM

The Sequence Diagram represents the interaction flow between system components.

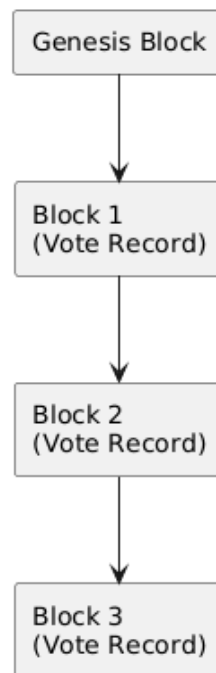


4.8 DATABASE / BLOCKCHAIN STRUCTURE

The system uses blockchain instead of a traditional centralized database.

Blockchain Structure

Blockchain Structure



The blockchain structure ensures immutability and transparency.

4.9 SYSTEM DESIGN

System design defines the complete structure and implementation approach of the project.

Modules of the System

1. Authentication Module

- Validates voter identity
- Prevents duplicate voting

2. Voting Module

- Displays candidates
- Records votes securely

3. Blockchain Module

- Creates and validates blocks
- Maintains blockchain integrity

4. Result Module

- Counts votes automatically
- Generates constituency-wise results

5. Commission Module

- Provides secure access to election authorities

CHAPTER - 5

CHAPTER - 5

Implementation & Testing

5.1 INTRODUCTION

Implementation represents a pivotal phase in the software development lifecycle, wherein the conceptualized architectural design is transformed into a functional operational framework. The **Secure Blockchain Voting System** is engineered utilizing the **Python** programming language and decentralized blockchain technology to deliver a secure, transparent, and tamper-proof substrate for electronic electoral processes.

The architectural framework integrates foundational blockchain principles, encompassing **SHA-256 cryptographic hashing**, **Proof-of-Work** consensus protocols, and immutable distributed ledgers to facilitate the secure archival of voting records. To enhance functional accessibility, the infrastructure incorporates diversified operational modalities, including a **Flask-based** web application, a **Tkinter** desktop GUI, and a command-line interface (CLI) modality.

The implementation phase delineates the development of core blockchain modules, user-centric web interfaces, electoral logic, and rigorous data validation protocols. The system is meticulously engineered to safeguard electoral integrity, mitigate duplicate voting practices, and facilitate the transparent aggregation of results. Such decentralized paradigms and secure blockchain implementations are increasingly prevalent within contemporary academic and research-oriented studies on digital democracy.

5.2 TECHNOLOGIES USED

The following technologies and tools are used in the implementation of the project:

Technology Stack	Functional Application
Python 3	Serves as the primary programming language for implementing core blockchain logic and system operations.
Flask	Utilized as the micro-web framework to facilitate the development of the browser-based voting interface.
Tkinter	Deployed to engineer the native desktop graphical user interface for enhanced cross-platform accessibility.
SHA-256	Implemented as the cryptographic hashing primitive to ensure the immutability and security of electoral data.
JSON	Used for deterministic data representation and standardized communication between distributed system modules.
Socket Programming	Facilitates robust client-server communication protocols within the decentralized architectural framework.
HTML/CSS/Bootstrap	Leveraged for the aesthetic design and responsive layout of the frontend web-based dashboard.
GitHub	Employed for rigorous source code management, version control, and collaborative developmental archival.
Pytest / Unittest	Integral testing frameworks utilized to validate systemic functionality through unit and integration testing.

Python is used because of its simplicity, rich libraries, and support for blockchain-related operations. Flask provides lightweight web application support, while Tkinter enables the creation of desktop-based voting interfaces.

5.3 PROJECT MODULES

The architectural framework is bifurcated into diverse functional modules to facilitate efficacious implementation, modular scalability, and streamlined systemic maintenance.

Fundamental Systemic Modules

1. **Blockchain Module**

- Facilitates the creation and administrative management of discrete blockchain blocks.
- Ensures the continuous preservation of cryptographic chain integrity.
- Provides a secure substrate for the immutable archival of electoral data.

2. **Voting Module**

- Manages the ingestion and processing of individual voter inputs.
- Executes rigorous validation protocols for unique voter identifications.
- Implements mechanisms to mitigate and prevent duplicate voting practices.

3. **Mining Module**

- Deploys the computational Proof-of-Work consensus algorithm.
- Authenticates candidate blocks prior to their definitive integration into the ledger.

4. **Web Interface Module**

- Delivers a lightweight, browser-centric electronic voting platform.
- Provides visual representations of the blockchain and real-time electoral outcomes.

5. **Desktop GUI Module**

- Engineers a standalone, native graphical user interface for enhanced accessibility.
- Facilitates operational functionality within standalone local environments.

6. **Commission View Module**

- Presents constituency-specific, cryptographically verified electoral results.
- Enables comprehensive administrative monitoring and procedural oversight.

7. **Testing Module**

- Executes systematic unit testing and functional verification protocols.
- Ensures the reliability and operational integrity of all blockchain-centric utilities.

5.4 BLOCKCHAIN IMPLEMENTATION

Blockchain technology constitutes the foundational architectural component of the engineered system. Within this decentralized substrate, each discrete vote is encapsulated and archived as a block comprising the following constituent metadata:

- **Block index**
- **Timestamp**
- **Vote data**
- **Previous hash**
- **Current hash**
- **Proof value**

The architectural framework leverages the **SHA-256** cryptographic primitive to generate deterministic hash signatures for each block. These blocks are sequentially interconnected via previous hash references, thereby facilitating inherent immutability and systemic data integrity.

Furthermore, the system implements a rigorous **Proof-of-Work** consensus mechanism, necessitating that each candidate block satisfy predefined cryptographic conditions prior to its definitive integration into the ledger. This blockchain-centric archival of electoral data ensures unparalleled tamper resistance and procedural transparency throughout the voting process.

Working of Blockchain

1. Submission of electoral input by the verified voter.
2. Encryption and obfuscation of voter identification leveraging **SHA-256**.
3. Archival of encrypted vote data within a newly initialized block.
4. Validation of the candidate block through the mining process.
5. Integration of the authenticated block into the distributed blockchain ledger.
6. Continuous verification and monitoring of blockchain integrity.

5.5 WEB APPLICATION IMPLEMENTATION

The web application is engineered utilizing the **Flask** framework, which provides a lightweight server-side substrate to facilitate seamless user interaction with the decentralized voting infrastructure via standard web browser modalities.

The integrated web interface encompasses the following functional components:

- **Voting Portal**
- **Blockchain Dashboard**
- **Election Results View**

- **Commission Login Interface**

The frontend utilizes **Bootstrap** to ensure a responsive and user-centric design across diverse devices. The Flask server serves as the orchestrator for administrative routing, electoral vote processing, real-time blockchain visualization, and robust API communication protocols.

Main Features of Web Interface

- Secure and transparent voting mechanism
- Real-time synchronization of blockchain records
- Intuitive candidate selection interface
- Dynamic aggregation of election statistics
- Responsive, mobile-friendly architectural design

The functional application is deployed within a local development environment, operating on port 5000 leveraging the Flask server utility.

5.6 GUI IMPLEMENTATION

The standalone desktop graphical user interface is engineered leveraging the **Tkinter** library, which serves as the native GUI primitive within the Python ecosystem. This architectural modality facilitates a robust and responsive desktop environment for electoral participation.

The functional GUI provides the following integrated utilities:

- Candidate selection facilitated through interactive radio button components
- Rigorous vote confirmation through automated modal dialogs
- Transparent administrative inspection of the decentralized blockchain ledger
- Authenticated access protocols for election commission oversight

The desktop interface serves as an efficacious modality for electoral operations within offline computational environments and standalone pedagogical demonstrations of blockchain-centric voting mechanisms.

Advantages of GUI Interface

- Enhanced operational simplicity and user-centric navigation

- Highly interactive and immersive electronic voting experience
- Robust, platform-independent execution across diverse operating systems
- Optimized and lightweight computational footprint

5.7 API IMPLEMENTATION

The architectural framework exposes a suite of RESTful API endpoints engineered to orchestrate secure voting operations and facilitate the deterministic retrieval of electoral data from the ledger.

Fundamental API Endpoints

API Endpoint	Functional Specification
<code>/api/cast-vote</code>	Orchestrates secure vote submission
<code>/api/blockchain</code>	Facilitates blockchain ledger retrieval
<code>/api/results</code>	Aggregates dynamic electoral outcomes
<code>/api/candidates/<constituency></code>	Fetches constituency-specific candidates
<code>/api/commission-view</code>	Provides verified administrative insights

These API interfaces facilitate deterministic data exchange utilizing the **JSON** format, ensuring seamless interoperability and synchronized communication between the frontend modalities and the decentralized backend architecture.

5.8 TESTING

Systemic testing is executed to authenticate functional correctness and cryptographic security. Diverse evaluative methodologies are deployed to verify blockchain integrity, electoral vote validation, and the operational stability of user interfaces.

The evaluative process scrutinizes the following parameters:

- Precision in electoral vote archival
- Rigorous blockchain validation protocols
- Mitigation of duplicate voting practices
- Deterministic generation of electoral results
- Accuracy of RESTful API responses

5.8.1 UNIT TESTING

Unit testing is implemented upon discrete functional modules to ensure that individual sub-routines operate according to specified logic.

The following systemic components are evaluated:

- Core blockchain functional utilities
- SHA-256 cryptographic hashing primitives
- Electoral vote validation logic
- Distributed server communication protocols

The **unittest** and **pytest** frameworks are leveraged to facilitate these testing objectives.

Execution command example:

```
Shell  
python3 -m pytest tests/
```

5.8.2 INTEGRATION TESTING

Integration testing verifies the deterministic communication and interoperability between disparate systemic modules.

The following architectural integrations are validated:

- Web interface connectivity with the blockchain module
- Desktop GUI synchronization with electoral voting logic
- Secure API communication protocols with the backend infrastructure
- Real-time blockchain validation during electoral operations

This phase ensures the cohesive operational synchronization of all integrated architectural layers.

5.8.3 SYSTEM TESTING

System testing is performed upon the comprehensive application framework to evaluate aggregate performance, reliability, and functional stability.

The infrastructure is rigorously tested for:

- End-to-end electronic voting lifecycle
- Real-time aggregation of electoral statistics
- Preservation of blockchain consistency and archival integrity
- Comprehensive validation of security protocols
- Functional efficacy of diverse user interfaces

The successful conclusion of these tests confirms that the **Secure Blockchain Voting System** remains operationally viable under varied environmental conditions.

5.9 SCREENSHOTS OF THE PROJECT

This section delineates a comprehensive visual representation of the diverse functional modules and user-centric interfaces engineered within the decentralized architectural framework.

The curated visual documentation encompasses the following systemic components:

- Home Page Dashboard
- Electronic Voting Interface
- Blockchain Ledger Dashboard
- Electoral Results Statistics
- Commission Authentication Portal
- Standalone GUI Application

These visual representations effectively demonstrate the operational efficacy, procedural transparency, and navigational aesthetics characterizing the implemented electronic voting solution.

5.9.1 HOME PAGE

The foundational home page dashboard facilitates seamless navigation across all primary functional modalities of the application, encompassing the electronic voting portal, real-time blockchain visualization, and dynamic electoral result aggregation.

Secure Blockchain Voting
Cast your vote securely on the blockchain

Voter Portal

Voter ID:
Enter ID starting with A, B, or C
Examples: A001, B123, C999

Constituency:
Enter voter ID first

Select Candidate:
Enter a valid voter ID first

Cast Vote

Reset Form

Dashboard

Quick Actions

View Blockchain

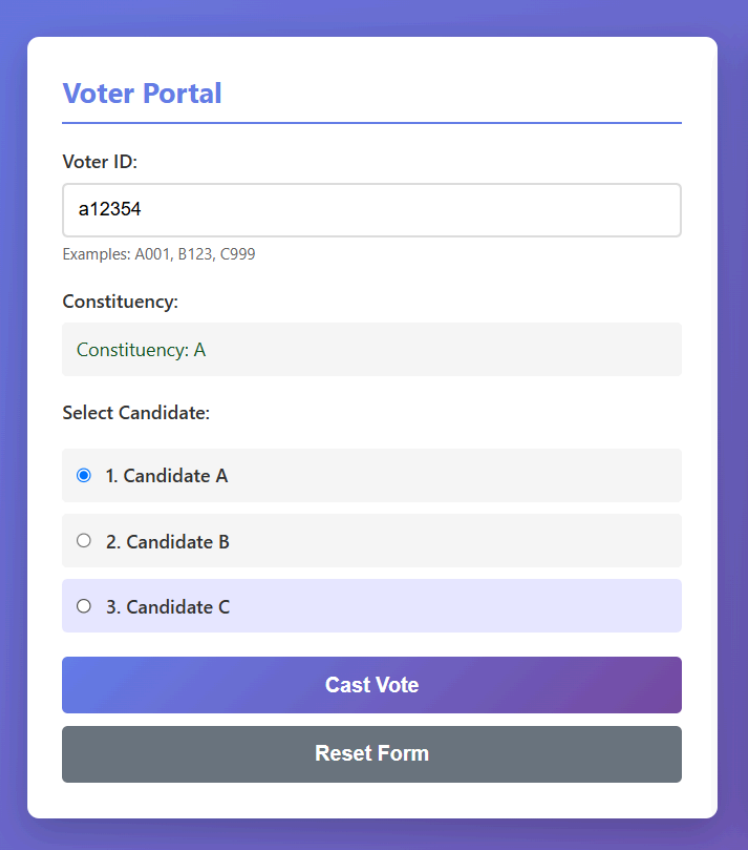
Election Results

Commission Login

System Status
Total Blocks: 1
Total Votes Cast: 0

5.9.2 VOTING INTERFACE

The integrated voting interface enables users to submit unique voter identifications, execute candidate selections based on constituency parameters, and commit electoral votes to the decentralized ledger through secure and transparent protocols.



Voter Portal

Voter ID:

Examples: A001, B123, C999

Constituency:

Constituency: A

Select Candidate:

☒ 1. Candidate A

☐ 2. Candidate B

☐ 3. Candidate C

Cast Vote

Reset Form

5.9.3 BLOCKCHAIN VIEW

This specialized interface delineates the comprehensive architectural structure of the blockchain, providing granular insights into discrete block indices, temporal metadata, Proof-of-Work values, and interconnected cryptographic hash sequences.

Blockchain Data

Block #	Timestamp	Votes Count	Proof	Previous Hash
1	5/27/2026, 9:44:16 AM	0	100	1...
2	5/27/2026, 9:46:26 AM	1	35293	beb3e5cb48b61b1c...
3	5/27/2026, 9:46:35 AM	1	35089	7617ab234ece797f...
4	5/27/2026, 9:46:44 AM	1	119678	f17a461d1a5178c3...

Close

5.9.4 RESULT DASHBOARD

The dynamic result dashboard presents constituency-specific electoral outcomes and automated vote aggregations in a deterministic tabular format, ensuring procedural transparency and real-time statistical accuracy.

Election Results

Constituency A

Candidate ID	Candidate Name	Votes
1	Candidate A	1
Total Votes:		1

Constituency B

Candidate ID	Candidate Name	Votes
2	Candidate Y	1

Close

5.9.5 COMMISSION LOGIN PAGE

The commission authentication portal facilitates secure and restricted access for authorized election officials, enabling the rigorous oversight and verification of cryptographically authenticated electoral results.



Comprehensive project source code and functional implementation specifications are archived and accessible via the official **GitHub** repository:

[E-voting System using Blockchain and Python GitHub Repository](#)

5.9.6 CODE

```
1 from flask import Flask, render_template, request, jsonify
2 from e_voting.blockchain import Blockchain
3
4 app = Flask(__name__)
5 blockchain = Blockchain()
6
7 @app.route('/')
8 def index():
9     return render_template('index.html')
10
11 @app.route('/api/candidates/<constituency>')
12 def get_candidates(constituency):
13     candidates = {
14         'A': {'1': 'Candidate A', '2': 'Candidate B', '3': 'Candidate C'},
15         'B': {'1': 'Candidate X', '2': 'Candidate Y', '3': 'Candidate Z'},
16         'C': {'1': 'Candidate D', '2': 'Candidate E', '3': 'Candidate F'},
17     }
18     return jsonify(candidates.get(constituency.upper(), {}))
19
20 @app.route('/api/cast-vote', methods=['POST'])
21 def cast_vote():
22     data = request.json
23     voter_id = data.get('voter_id', '').strip()
24     constituency = voter_id[:1].upper() if voter_id else ''
25     candidate_id = data.get('candidate_id', '')
26
27     if not voter_id:
28         return jsonify({'success': False, 'error': 'Voter ID is required'}), 400
29
30     if voter_id in blockchain.voter_ids:
31
32         def cast_vote():
33             if voter_id in blockchain.voter_ids:
34
35                 if constituency not in ['A', 'B', 'C']:
36                     return jsonify({'success': False, 'error': 'Invalid voter ID format'}), 400
37
38                 if not candidate_id:
39                     return jsonify({'success': False, 'error': 'Candidate must be selected'}), 400
40
41                 vote_data = {'constituency': constituency, 'candidate_id': candidate_id}
42                 blockchain.new_vote(voter_id, vote_data)
43
44                 # Mine the block and add it to the chain
45                 last_block = blockchain.chain[-1] if blockchain.chain else None
46                 last_proof = last_block.get('proof', 100) if last_block else 100
47                 proof = blockchain.proof_of_work(last_proof)
48                 previous_hash = blockchain.hash(last_block) if last_block else '1'
49                 blockchain.new_block(proof, previous_hash)
50
51                 blockchain.increment_candidate_votes(constituency, candidate_id)
52                 blockchain.voter_ids.add(voter_id)
53
54                 return jsonify({
55                     'success': True,
56                     'message': 'Vote cast successfully'
57                 })
58
59 @app.route('/api/blockchain')
60 def get_blockchain():
61     return jsonify(blockchain.view_blockchain())
62
63 @app.route('/api/results')
64 def get_results():
65     results = {}
66     for constituency, votes in blockchain.candidates.items():
67         results[constituency] = []
68         for candidate_id, info in votes.items():
69             results[constituency].append({
70                 'name': info['name'],
71                 'id': candidate_id,
72                 'votes': info['votes']
73             })
74     return jsonify(results)
75
76 @app.route('/api/commission-view', methods=['POST'])
77 def commission_view():
78     data = request.json
79     login_id = data.get('login_id', '')
80     constituency = data.get('constituency', '').upper()
81
82     result = blockchain.election_commission_view(login_id, constituency)
83
84     if 'error' in result:
85         return jsonify({'success': False, 'error': result['error']}), 403
86
87     return jsonify({'success': True, 'data': result})
88
89 if __name__ == '__main__':
90     app.run(debug=False, host='0.0.0.0', port=5000)
```

```

1 import hashlib
2 import json
3 import time
4
5 class Blockchain:
6     def __init__(self):
7         self.chain = []
8         self.current_votes = []
9         self.nodes = set()
10        self.voter_ids = set() # Set to keep track of used voter IDs
11        self.candidates = {} # {constituency: {candidate_id: {'name': candidate_name, 'votes': vote_count}}}
12
13        # Genesis block
14        self.new_block(proof=100, previous_hash='1')
15
16    def new_block(self, proof, previous_hash):
17        block = {
18            'index': len(self.chain) + 1,
19            'timestamp': time.time(),
20            'votes': self.current_votes,
21            'proof': proof,
22            'previous_hash': previous_hash,
23        }
24        self.current_votes = []
25        self.chain.append(block)
26        return block
27
28    def hash(self, block):
29        block_string = json.dumps(block, sort_keys=True).encode()
30        return hashlib.sha256(block_string).hexdigest()
31
32    def proof_of_work(self, last_proof):
33        proof = 0
34        while self.valid_proof(last_proof, proof) is False:
35            proof += 1
36        return proof
37
38    def valid_proof(self, last_proof, proof):
39        guess = f'{last_proof}{proof}'.encode()
40        guess_hash = hashlib.sha256(guess).hexdigest()
41        return guess_hash[:4] == '0000'
42
43    def new_vote(self, person_id, vote):
44        vote_entry = {
45            'voter_id': hashlib.sha256(str(person_id).encode()).hexdigest(),
46            'vote': vote,
47        }
48        self.current_votes.append(vote_entry)
49        return vote_entry
50
51    def confirm_vote(self):
52        confirmation = input('Do you want to cast your vote? (yes/no): ').lower()
53        return confirmation == 'yes'
54
55    def increment_candidate_votes(self, constituency, candidate_id):
56        if constituency not in self.candidates:
57            self.candidates[constituency] = {}
58
59        if constituency == 'A':
60
61            elif constituency == 'B':
62                candidate_names = ['Candidate X', 'Candidate Y', 'Candidate Z']
63            else: # Default to 'C' constituency
64                candidate_names = ['Candidate D', 'Candidate E', 'Candidate F']
65
66            if candidate_id not in self.candidates[constituency]:
67                self.candidates[constituency][candidate_id] = {'name': candidate_names[int(candidate_id) - 1], 'votes': 1}
68            else:
69                self.candidates[constituency][candidate_id]['votes'] += 1
70
71    def election_commission_view(self, login_id, constituency):
72        if login_id != '55':
73            return {'error': 'Access denied. Invalid login ID.'}
74
75        if constituency not in self.candidates:
76            return {'error': f'No votes recorded for constituency: {constituency}'}
77
78        votes = self.candidates[constituency]
79        results = []
80        total_votes = 0
81
82        for candidate_id, candidate_info in votes.items():
83            results.append({
84                'candidate_id': candidate_id,
85                'name': candidate_info['name'],
86                'votes': candidate_info['votes']
87            })
88            total_votes += candidate_info['votes']

```

```

90         return {
91             'constituency': constituency,
92             'total_votes': total_votes,
93             'candidates': results
94         }
95
96     def view_blockchain(self):
97         return self.chain
98
99
100 if __name__ == '__main__':
101     # Example usage:
102     blockchain = Blockchain()
103
104     while True:
105         print('\nOptions:')
106         print('1. Cast Vote')
107         print('2. Election Commission View')
108         print('3. View Blockchain')
109         print('4. Exit')
110
111         choice = input('Enter your choice (1/2/3/4): ')
112
113         if choice == '1':
114             voter_id = input('Enter your voter ID: ')
115             constituency = voter_id[:1].upper()
116
117             if voter_id in blockchain.voter_ids:
118                 if voter_id in blockchain.voter_ids:
119                     print('You have already casted a vote. Exiting...')
120                     break
121
122             if constituency not in ['A', 'B', 'C']: # Add more constituencies as needed
123                 print('Invalid constituency. Please try again.')
124                 continue
125
126             candidates = {}
127
128             if constituency == 'A':
129                 candidates = {
130                     '1': 'Candidate A',
131                     '2': 'Candidate B',
132                     '3': 'Candidate C'
133                 }
134             elif constituency == 'B':
135                 candidates = {
136                     '1': 'Candidate X',
137                     '2': 'Candidate Y',
138                     '3': 'Candidate Z'
139                 }
140             else: # Default to 'C' constituency
141                 candidates = {
142                     '1': 'Candidate D',
143                     '2': 'Candidate E',
144                     '3': 'Candidate F'
145                 }
146
147             print(f'\nCandidates for constituency {constituency}:')
148
149             for candidate_id, candidate_name in candidates.items():
150                 print(f'{candidate_id}: {candidate_name}')
151
152             vote_casted = False
153             while not vote_casted:
154                 candidate_id = input('Enter the candidate ID you want to vote for (or type "exit" to finish): ')
155                 if candidate_id.lower() == 'exit':
156                     break
157                 if candidate_id in candidates:
158                     if blockchain.confirm_vote():
159                         blockchain.new_vote(voter_id, {'constituency': constituency, 'candidate_id': candidate_id})
160                         blockchain.increment_candidate_votes(constituency, candidate_id)
161                         blockchain.voter_ids.add(voter_id) # Mark voter ID as used
162                         print('Vote casted successfully!')
163                         vote_casted = True
164                     else:
165                         print('Vote cancelled.')
166                 else:
167                     print('Invalid candidate ID. Please try again.')
168
169             elif choice == '2':
170                 print(blockchain.election_commission_view())
171
172             elif choice == '3':
173                 print('Blockchain:', blockchain.view_blockchain())
174
175             elif choice == '4':
176                 print('Exiting...')
177                 break

```

```

1  ##client
2  import socket
3  import hashlib
4
5  # Dictionary to store voters and the candidates they have voted for
6  voters_choices = {}
7
8  # Dictionary to store candidates, their SHA-256 hashes, and their political parties
9  candidates_info = {
10     'Candidate 1': {'hash': hashlib.sha256('Candidate 1'.encode()).hexdigest(), 'party': 'Party A'},
11     'Candidate 2': {'hash': hashlib.sha256('Candidate 2'.encode()).hexdigest(), 'party': 'Party B'},
12     'Candidate 3': {'hash': hashlib.sha256('Candidate 3'.encode()).hexdigest(), 'party': 'Party A'},
13 }
14
15 def encrypt_candidate(candidate_name):
16     return hashlib.sha256(candidate_name.encode()).hexdigest()
17
18 def display_candidate_list():
19     print("List of Candidates and their Political Parties:")
20     for candidate, info in candidates_info.items():
21         print(f"{candidate} ({info['party']})")
22
23 def main():
24     host = '127.0.0.1' # Server's IP address or hostname
25     port = 8088
26
27     client_socket = socket.socket()
28     client_socket.connect((host, port))
29

```

```

31     display_candidate_list()
32
33     while True:
34         candidate_name = input("Enter the candidate's name: ")
35
36         if candidate_name.lower() == 'exit':
37             break
38
39         if candidate_name in candidates_info:
40             if client_socket.getpeername() in voters_choices:
41                 print("You have already voted for a candidate. Please choose another candidate.")
42             else:
43                 encrypted_candidate = encrypt_candidate(candidate_name)
44                 client_socket.send(encrypted_candidate.encode())
45                 # Add the voter's choice to the dictionary
46                 voters_choices[client_socket.getpeername()] = candidate_name
47                 print("Your vote for", candidate_name, "has been cast.")
48         else:
49             print("Invalid candidate. Please choose a candidate from the list.")
50
51     client_socket.close()
52
53 if __name__ == "__main__":
54     main()

```

```

1  ##server
2  import socket
3  import hashlib
4  import json
5
6  # List of valid candidates with their SHA-256 hashes
7  valid_candidates = {
8     hashlib.sha256('Candidate 1'.encode()).hexdigest(): 'Candidate 1',
9     hashlib.sha256('Candidate 2'.encode()).hexdigest(): 'Candidate 2',
10    hashlib.sha256('Candidate 3'.encode()).hexdigest(): 'Candidate 3',
11 }
12
13 class Blockchain:
14     def __init__(self):
15         self.chain = []
16         self.current_votes = [] # Initialize current_votes
17         self.create_block(previous_hash='1') # Genesis block
18
19     def create_block(self, previous_hash):
20         block = {
21             'index': len(self.chain) + 1,
22             'votes': self.current_votes,
23             'previous_hash': previous_hash,
24         }
25         self.current_votes = [] # Reset current votes
26         self.chain.append(block)
27
28     def hash(self, block):
29         return hashlib.sha256(json.dumps(block, sort_keys=True).encode()).hexdigest()
30

```

CHAPTER - 6

CHAPTER - 6

RESULTS & FUTURE SCOPE

6.1 PROJECT RESULTS

The engineered **Secure Blockchain Voting System** was successfully architected and implemented utilizing the Python programming language and decentralized blockchain technology. This infrastructure delivers a secure, transparent, and tamper-proof electronic voting substrate, effectively mitigating duplicate voting practices and ensuring the overarching integrity of the electoral process.

The project successfully demonstrates the pragmatic application of foundational blockchain architectural concepts, encompassing:

- **SHA-256 cryptographic hashing**
- **Proof-of-Work consensus mechanism**
- **Immutable blockchain ledger archival**
- **Secure electoral vote storage**
- **Rigorous blockchain validation protocols**

The developed infrastructure facilitates diversified operational modalities, including:

- Flask-based Web Application
- Tkinter Desktop GUI
- Command-Line Interface (CLI)

The system empowers voters to cast selections securely while maintaining confidentiality through the encryption of voter identifications. Electoral results are dynamically aggregated and presented constituency-wise in a transparent and deterministic format.

The decentralized blockchain architecture ensures that once voting records are committed to the ledger, they remain impervious to unauthorized modification or retrospective deletion. This mechanism significantly enhances public trust and the perceived reliability of digital electoral processes.

Major Achievements of the Project

- Successful implementation of a blockchain-centric voting paradigm
- Comprehensive mitigation of duplicate voting practices
- Real-time capabilities for continuous blockchain verification
- Secure vote archival leveraging SHA-256 cryptographic primitives
- Transparent and deterministic generation of election outcomes
- Integrated support for multi-platform operational interfaces
- Optimized, lightweight, and user-centric systemic design

The project fulfills its foundational objective of engineering a secure and decentralized electronic voting framework utilizing robust blockchain technology.

6.2 ADVANTAGES OF THE SYSTEM

The proposed **Secure Blockchain Voting System** offers several paramount advantages over conventional electoral methodologies.

Fundamental Systemic Advantages

1. Security

The architecture leverages SHA-256 cryptographic hashing and decentralized blockchain technology to fortify electoral data against unauthorized manipulation or adversarial access.

2. Transparency

All electoral selections are archived within a distributed blockchain ledger that facilitates public verification, thereby fostering systemic trust and procedural accountability.

3. Mitigation of Duplicate Voting

Systemic protocols rigorously validate each unique voter identification, ensuring that each participant is restricted to a singular electoral submission.

4. Immutability

Upon the successful integration of a vote into a blockchain block, the record becomes inherently immutable, precluding retrospective alteration or unauthorized deletion.

5. Decentralization

The blockchain-centric architecture reduces reliance on centralized administrative authorities, effectively eliminating single-point failure vulnerabilities.

6. Efficient Result Generation

Electoral outcomes are computed through automated algorithmic aggregation, significantly accelerating the tallying process while eliminating human inaccuracies.

7. User-Centric Modalities

The project delivers intuitive Web, Desktop GUI, and CLI interfaces to ensure comprehensive accessibility for diverse user demographics.

8. Auditability

The distributed blockchain ledger serves as a verifiable substrate for post-election auditing, ensuring unparalleled transparency throughout the electoral lifecycle.

6.3 LIMITATIONS

While the engineered system offers significant enhancements to security and transparency, certain operational and technical limitations remain prevalent.

Evaluative Constraints of the System

- The current prototype is primarily engineered for academic research and pedagogical demonstration purposes.
- The implementation utilizes a standardized commission identification, necessitating more advanced authentication paradigms for large-scale deployment.
- Proof-of-Work mining operations may require significant computational resources during high-volume electoral processes.
- The blockchain architecture is presently implemented within local environments rather than distributed across global network nodes.
- A fundamental internet dependency characterizes the functional accessibility of the web-based interface.
- The current framework accommodates a restricted number of constituencies and electoral candidates.

- Advanced cryptographic mechanisms, such as RSA or ECC, remain outside the scope of the current implementation.

These identified constraints provide the impetus for future developmental iterations and systemic optimizations.

6.4 FUTURE ENHANCEMENTS

The **Secure Blockchain Voting System** can be significantly fortified through the integration of advanced security paradigms and scalable architectural features.

Potential Systemic Enhancements

1. Advanced Authentication Modalities

Implementation of biometric verification, multi-factor OTP protocols, or national identification-centric (Aadhaar) authentication to fortify voter identity management.

2. Distributed Network Deployment

Migration of the blockchain ledger to a distributed network of peer-to-peer nodes to enhance decentralization and systemic fault tolerance.

3. Smart Contract Orchestration

Utilization of Ethereum-based smart contracts to automate electoral validation and deterministic result tallying.

4. Enhanced Database Interoperability

Integration with robust database architectures, such as PostgreSQL or MongoDB, to optimize scalability and complex data management.

5. Mobile Platform Development

Engineering of native Android and iOS applications to facilitate secure and accessible remote electronic voting.

6. Real-Time Communication Protocols

Deployment of WebSocket technology to provide live updates and automated notifications during the electoral process.

7. Sophisticated Cryptographic Primitives

Integration of RSA, ECC, or Zero-Knowledge Proof (ZKP) methodologies to provide superior security and voter anonymity.

8. Cloud Infrastructure Deployment

Hosting of the application framework on high-availability cloud platforms such as AWS, Azure, or Google Cloud for global scalability.

9. Multi-Electoral Support

Extension of systemic support to facilitate multiple concurrent elections and larger voter populations within a unified framework.

These prospective enhancements will elevate the system toward production-grade readiness, making it suitable for functional implementation in real-world governmental elections.

6.5 CONCLUSION

The engineered **Secure Blockchain Voting System** serves as a functional demonstration of the efficacy of blockchain technology in revolutionizing contemporary electronic voting paradigms by enhancing security, transparency, and systemic trust.

Conventional electoral infrastructures frequently grapple with foundational issues such as vote manipulation, opaque administrative processes, centralized authority, and duplicate voting. The proposed architectural framework effectively mitigates these challenges through the strategic integration of decentralized blockchain technology, SHA-256 cryptographic hashing, and rigorous Proof-of-Work consensus mechanisms.

The developed infrastructure facilitates the secure archival of electoral selections within blocks that attain inherent immutability upon validation. Furthermore, the system delivers diversified operational interfaces, including Web, Desktop GUI, and CLI modalities, ensuring unparalleled functional flexibility and user accessibility.

The research effectively proves that blockchain technology can provide a robust and transparent substrate for future digital electoral processes. Although the current implementation is designed for academic and pedagogical purposes, it establishes a foundational architectural blueprint for engineering advanced and globally scalable blockchain-centric voting systems.

In summary, the proposed solution successfully achieves its primary research objectives and underscores the practical utility of blockchain technology in developing secure electronic voting applications.

REFERENCES

1. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Bitcoin.org. [Bitcoin Whitepaper Archive](#)
2. Jafar, U., & Aziz, N. A. (2021). *Blockchain for Electronic Voting System—Review and Open Research Challenges*. *Sensors*, 21(17). [Scholarly Review on Blockchain E-Voting](#)
3. Hajian Berenjestanaki, M., et al. (2023). *Blockchain-Based E-Voting Systems: A Technology Review*. *Electronics*, 13(1). [MDPI Technology Review Ledger](#)
4. Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson Education.
5. Antonopoulos, A. M. (2017). *Mastering Bitcoin* (2nd ed.). O'Reilly Media.
6. Swan, M. (2015). *Blockchain: Blueprint for a New Economy*. O'Reilly Media.
7. Emami, A., et al. (2023). *A Scalable Decentralized Privacy-Preserving E-Voting Scheme Using Blockchain*. *Journal of Information Security and Applications*. [Research on Decentralized Privacy](#)
8. Yan, S. (2022). *Analysis on Blockchain Consensus Mechanism Based on Proof of Work and Proof of Stake*. arXiv. [Consensus Protocol Comparative Analysis](#)

For Access Our Project Scan It :

